6.6 View a single product

Our product list and a new detail page need the same collection. Move the sample objects from productList into a property initialized by ProductController's constructor. Then connect `?action=product&id=1` to a public method that displays one product. This extends the chapter's architecture with a second action in the same specialized controller.

Replace ProductController

Replace `src/Tutorials/Class6/ProductController.php` with:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

class ProductController extends Controller
{
    /** @var array<int, Product> */
    private array $products;

    public function __construct()
    {
        parent::__construct();

        $this->products = [
            1 => new Product('Study notebook', 4.50),
            2 => new Product('Desk organizer', 12.00),
            3 => new Product('Pencil set', 3.25),
        ];
    }

    public function productList(): void
    {
        $template = 'products.html.twig';
        $args = ['products' => $this->products];
        echo $this->twig->render($template, $args);
    }

    public function showProduct(int $id): void
    {
        $product = $this->products[$id] ?? null;
        if ($product === null) {
            $this->productNotFound();
            return;
        }

        $template = 'product.html.twig';
        $args = ['product' => $product];
        echo $this->twig->render($template, $args);
```

```
    }

    public function productNotFound(): void
    {
        http_response_code(404);
        echo $this->twig->render('product-not-found.html.twig', []);
    }
}
```

The child now defines a constructor, so PHP no longer automatically runs the inherited constructor for it. `parent::__construct()` explicitly initializes Twig before this constructor creates the products. Omitting that call would leave the inherited twig property uninitialized when either action tries to render.

The array keys 1, 2, and 3 are our sample product IDs. Keeping explicit keys makes the same ID available to the list links and detail lookup; a separate ID property is unnecessary for this example. The docblock describes the array's key and element types for readers and editors; PHP enforces the array type itself.

Both actions now read `$this->products`. This shares data within the controller object for a request, not across browser requests. Each new request constructs a new controller and rebuilds the sample collection. No session or database is involved. Product's property hooks still validate every object created here.

Add the detail and not-found views

Create `templates/tutorials/class6/product.html.twig`:

```
{% extends 'base.html.twig' %}
{% block pageTitle %}{{ product.name }}{% endblock %}
{% block productsLink %}active{% endblock %}
{% block main %}
    <p>Price: ${{ product.price|number_format(2) }}</p>
    <p><a href="index.php?action=products">Back to products</a></p>
{% endblock %}
```

The singular product context key contains one object. Its name appears in both the tab title and h1 through the existing pageTitle block. Products stays active in navigation because this page belongs to the product section.

Create `templates/tutorials/class6/product-not-found.html.twig`:

```
{% extends 'base.html.twig' %}
{% block pageTitle %}Product not found{% endblock %}
{% block productsLink %}active{% endblock %}
{% block main %}
    <p>We could not find that product. Choose an item from the product list.</p>
    <p><a href="index.php?action=products">Back to products</a></p>
{% endblock %}
```

The error method sets HTTP status 404 before rendering. Its template needs no product context. Returning immediately after the error in showProduct prevents a second render with a missing product.

Replace Application to add the detail route

Replace `src/Tutorials/Class6/Application.php` with:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class6;

class Application
{
    public function run(): void
    {
        $defaultController = new DefaultController();
        $productController = new ProductController();

        $action = $_GET['action'] ?? 'home';
        if (!is_string($action)) {
            $action = 'home';
        }

        switch ($action) {
            case 'product':
                $rawId = $_GET['id'] ?? null;
                $id = is_string($rawId)
                    ? filter_var($rawId, FILTER_VALIDATE_INT, ['options' =>
['min_range' => 1]])
                    : false;

                if ($id === false) {
                    $productController->productNotFound();
                    break;
                }

                $productController->showProduct($id);
                break;
            case 'products':
                $productController->productList();
                break;
            case 'contact':
                $defaultController->contact();
                break;
            case 'home':
            default:
                $defaultController->home();
        }
    }
}
```

Application checks the request's shape: id must be a string representing a positive integer. filter_var returns a validated integer or false, so use the strict false comparison. Do not simply cast the input to int: a value such as `1abc` could become 1 without being a valid ID. An array-shaped input also fails the string check.

The front controller validates the parameter before calling showProduct with an int. ProductController owns the collection, so it checks whether that ID exists. Missing, malformed, and nonexistent product IDs all show our 404 view. Unknown actions still follow the existing Home fallback; that is a separate routing decision.

Replace the list template with links

Replace `templates/tutorials/class6/products.html.twig` with:

```
{% extends 'base.html.twig' %}
{% block pageTitle %}Products{% endblock %}
{% block productsLink %}active{% endblock %}
{% block main %}
    <p>Supplies for your study space. Select a product to view its page.</p>
    <dl>
        {% for id, product in products %}
            <dt><a href="index.php?action=product&amp;id={{ id }}">{{ product.name
}}</a></dt>
            <dd>${{ product.price|number_format(2) }}</dd>
        {% else %}
            <dt>Availability</dt>
            <dd>No products available.</dd>
        {% endfor %}
    </dl>
{% endblock %}
```

The two-variable loop exposes the array key as id and the value as product. The link uses singular action=product for a detail page; the shared navigation uses plural action=products for the list. `&` is the HTML spelling of the query separator; the browser sends an ordinary ampersand in the request. Keep base.html.twig and the entry script unchanged.

Verify both actions

1. Open Products and follow all three product links. Expect the selected name as the title and heading, its matching price, and a working Back to products link.
2. Open `?action=product&id=2` directly. Expect Desk organizer and $12.00 with the Products navigation link active.
3. Try `?action=product`, `?action=product&id=abc`, `?action=product&id=1abc`, `?action=product&id=0`, `?action=product&id=-1`, `?action=product&id[]=1`, and `?action=product&id=999`. Each should show Product not found. In the browser's Network panel, verify the document response has status 404 and contains only one page.
4. Temporarily set `$this->products = [];` in the constructor. The list should show its empty state, and a detail request should show the 404 page. Restore the three objects afterward.
5. Recheck Home, Contact, and the unknown-action fallback.